

Search-Based Exercises on Selecting Python Data Structures and Basic Algorithms

1 Learning Objectives

After completing these exercises, students should be able to:

- examine raw data and identify the operations that must be supported;
- select an appropriate Python data structure for the required operations;
- select a suitable basic algorithm or standard-library tool;
- find relevant Python documentation using search engines or AI assistants;
- explain briefly why a selected solution is appropriate.

2 General Instructions

Each exercise begins with raw data and a list of required operations. Students must not assume that the original representation is the best representation.

For each exercise:

1. identify the important properties of the raw data;
2. identify the operations that must be performed;
3. select a suitable Python data structure or algorithm;
4. identify the relevant Python function, method, class, or module;
5. explain the choice in approximately three to five sentences;
6. provide at least one reference source.

Use the following response template:

Search keywords or prompts:

Important properties of the raw data:

Required operations:

Selected Python data structure or algorithm:

Python function, method, class, or module:

Short explanation:

Reference:

3 Part I: Selecting Suitable Python Data Structures

3.1 Exercise 1: Customer Email Records

Raw Data

```
raw_emails = [  
    "alice@example.com",  
    "bob@example.com",  
    "alice@example.com",  
    "charlie@example.com",  
    "bob@example.com"  
]
```

Required Operations and Search Tasks

The program must:

1. remove duplicate email addresses;
2. test quickly whether an email address has already appeared;
3. find email addresses shared by two customer lists.

Search for a Python data structure that directly supports these operations. Identify its main methods or operators.

3.2 Exercise 2: Product Information

Raw Data

```
raw_products = [  
    ["P01", "Laptop", 1200],  
    ["P02", "Mouse", 25],  
    ["P03", "Keyboard", 80],  
    ["P04", "Monitor", 300]  
]
```

Required Operations and Search Tasks

The program must:

1. find a product quickly from its product ID;
2. update the price of a selected product;
3. add or remove a product;
4. iterate over product IDs and their associated information.

Search for a Python data structure that is more suitable than repeatedly scanning the raw list.

3.3 Exercise 3: Repeated Words in Comments

Raw Data

```
raw_words = [  
    "good", "fast", "good", "delivery",  
    "fast", "helpful", "good"  
]
```

Required Operations and Search Tasks

The program must:

1. count how many times each word appears;
2. obtain the frequency of a selected word;
3. return the two most frequent words.

Search for a Python standard-library data structure designed specifically for frequency counting.

3.4 Exercise 4: Products Grouped by Category

Raw Data

```
raw_products = [  
    ["Laptop", "Electronics"],  
    ["Mouse", "Electronics"],  
    ["Notebook", "Stationery"],  
    ["Pen", "Stationery"],  
    ["Keyboard", "Electronics"]  
]
```

Required Operations and Search Tasks

The program must transform the raw data into a structure of the form:

`category -> collection of products`

It must also allow new products to be added to a category without manually checking whether the category already exists.

Search for a suitable Python data structure and a standard-library class that automatically creates an empty collection for a missing key.

3.5 Exercise 5: Print Jobs

Raw Data

```
raw_jobs = [  
    "Job 1",  
    "Job 2",  
    "Job 3",  
    "Job 4"  
]
```

Required Operations and Search Tasks

New jobs arrive continuously. The program must:

1. add each new job at the end;
2. process the oldest job first;
3. remove jobs efficiently from the front;
4. follow the first-in, first-out principle.

Search for a suitable Python standard-library data structure.

3.6 Exercise 6: Undo History

Raw Data

```
raw_actions = [  
    "Type title",  
    "Insert image",  
    "Change font",  
    "Delete paragraph"  
]
```

Required Operations and Search Tasks

The program must:

1. add a new action to the history;
2. undo the most recent action first;
3. inspect the most recent action without removing it;
4. follow the last-in, first-out principle.

Search for a simple Python data structure and the methods needed to implement these operations.

4 Part II: Selecting Suitable Basic Python Algorithms

4.1 Exercise 7: Ranking Products

Raw Data

```
raw_products = [  
    ["Laptop", 1200],  
    ["Mouse", 25],  
    ["Keyboard", 80],  
    ["Monitor", 300]  
]
```

Required Operations and Search Tasks

The program must:

1. rank products by price from low to high;
2. rank products by price from high to low;
3. rank products alphabetically by name;
4. optionally preserve the original raw data.

Search for the basic Python sorting tools that directly support these operations. Explain the roles of the sorting key and descending-order option.

4.2 Exercise 8: Searching and Inserting in Sorted Prices

Raw Data

```
raw_prices = [15, 20, 25, 31, 38, 45, 52, 60]
```

Required Operations and Search Tasks

The data is already sorted. The program must:

1. find the position of the value 38;
2. determine where the value 35 should be inserted;
3. insert 35 while preserving sorted order.

Search for a Python standard-library module that implements binary-search-based operations for sorted lists.

4.3 Exercise 9: Selecting the Largest Sales Values

Raw Data

```
raw_sales = [120, 95, 180, 150, 210, 130, 175]
```

Required Operations and Search Tasks

The program must return only the three largest values.

Search for:

1. a solution based on sorting;
2. a Python standard-library algorithm designed to select the largest k elements directly.

Compare the two approaches when the list is very large but k is small.

4.4 Exercise 10: Combining Sorted Timestamp Data

Raw Data

```
raw_times_a = [2, 8, 15, 21]
raw_times_b = [3, 9, 12, 25]
```

Required Operations and Search Tasks

Both input collections are already sorted. The program must:

1. combine them into one sorted sequence;
2. avoid sorting the complete combined data again;
3. process the input sequences efficiently.

Search for a Python standard-library algorithm that merges already sorted iterables.

5 Assessment Rubric

Criterion	Points
Relevant search keywords or prompts	1
Correct identification of required operations	2
Suitable data structure or algorithm	2
Appropriate Python function, class, method, or module	2
Clear explanation of the choice	2
Reliable reference source	1
Total	10

Alternative solutions may receive full credit when they support the required operations, are clearly explained, and are supported by a reliable source.